

MANUAL DE USUARIO

Proxy inverso L7

Instalación, configuración y operación

Guía práctica para compilar, configurar, poner en marcha, supervisar y diagnosticar el proxy.

Versión	2.0.0 (Rust)
Fecha	1 de octubre de 2026
Binario	<code>target/release/proxy</code>
Referencia	<code>proxy.toml</code> (comentado) · <code>SPEC.md</code>

Índice

1. Qué es y qué hace
2. Instalación
3. Primeros pasos (demo en 5 minutos)
4. Línea de comandos y señales
5. Referencia de configuración
 - [global]
 - [timeouts]
 - [[frontend]] y rutas
 - [[backend]] y health
6. Enrutamiento por dominio
7. HTTPS y certificados
8. Balanceo de carga
9. Health checks
10. Recarga en caliente
11. Supervisión: estadísticas y logs
12. Resolución de problemas
13. Herramientas de prueba y benchmark
14. Rendimiento y comparativas

1. Qué es y qué hace

El proxy se sitúa delante de tus servidores web. Escucha en uno o varios puertos (HTTP o HTTPS), mira el dominio que pide cada cliente y reenvía la petición a uno de los servidores del grupo configurado para ese dominio.

- **Varios puertos de entrada** (frontends), cada uno con sus propias reglas.
- **Enrutamiento por dominio**: exacto (`api.test`), comodín (`*.example.com`) y ruta por defecto.
- **HTTPS** con varios certificados por puerto, elegidos por el nombre que pide el cliente (SNI).
- **Balanceo**: round robin, por pesos, por menos conexiones o según la carga que informa cada servidor.
- **Tolerancia a fallos**: detecta servidores caídos y los saca del reparto.
- **Recarga sin cortes**: basta con guardar el fichero de configuración.
- HTTP/1.1 completo: conexiones persistentes, pipelining, subidas grandes y WebSocket.

Plataformas: **Linux** (verificado). macOS y BSD están contemplados en el código, pero aún no se han compilado ni probado en esas plataformas. Windows no está soportado.

Esta es la versión 2.0, escrita en **Rust**. Se configura y se maneja exactamente igual que las versiones anteriores en C: el mismo fichero de configuración, las mismas opciones, las mismas señales y las mismas estadísticas.

2. Instalación

2.1 Requisitos

Para	Necesitas
Compilar	Rust estable con edición 2024 (probado con 1.98), con <code>cargo</code> . Para el proveedor TLS por defecto (<code>aws-lc-rs</code>): compilador de C y <code>cmake</code>
Tests de integración	<code>curl</code> , <code>jq</code> , <code>socat</code> , <code>openssl</code>
Benchmark	<code>wrk</code>

Rust se instala con `rustup`. El resto, en Ubuntu/Debian:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
sudo apt-get install build-essential cmake curl jq socat openssl wrk
```

En macOS (Homebrew): `brew install cmake wrk jq socat` . Sin `cmake` se puede compilar con el proveedor `ring` :
`cargo build --release --no-default-features -F ring` .

2.2 Compilar

```
cargo build --release
cargo test                # opcional: 43 tests unitarios
```

Se generan tres programas en `target/release/` : `proxy` (el proxy), `test_backend` (servidor de pruebas) y `ws_probe` (cliente WebSocket de pruebas). El perfil `release` ya activa LTO. Para instalar el proxy:

```
cargo install --path . --bin proxy    # instala 'proxy' en ~/.cargo/bin
sudo install -m 755 target/release/proxy /usr/local/bin/  # o a mano
```

3. Primeros pasos (demo en 5 minutos)

El proyecto trae un generador de configuración de prueba con su propia autoridad certificadora.

```
# 1. Configuración + certificados de prueba en ./demo
./tests/gen_config.sh demo

# 2. Unos servidores de ejemplo
./target/release/test_backend --port 19001 --name api-1 &
./target/release/test_backend --port 19002 --name api-2 &
./target/release/test_backend --port 19004 --name web-1 &

# 3. El proxy
./target/release/proxy -c demo/proxy.toml
```

En otra terminal:

```
curl -H 'Host: api.test' http://127.0.0.1:18080/
hello from api-1 path=/                (+ una línea de relleno hasta 100 B)
curl -H 'Host: api.test' http://127.0.0.1:18080/
hello from api-2 path=/                (round robin)
curl -H 'Host: www.example.com' http://127.0.0.1:18080/
hello from web-1 path=/                (comodin *.example.com)

# HTTPS, verificando el certificado con la CA de la demo
curl --cacert demo/certs/ca.pem --resolve api.test:18443:127.0.0.1 https://api.test:18443/
```

Sin tocar `/etc/hosts`. `curl --resolve o -H 'Host: ...'` bastan para probar dominios. Si quieres usar los nombres en el navegador: `sudo ./tests/hosts.sh add` (se deshace con `remove`).

Para parar: `Ctrl+C` en la terminal del proxy. Hace una parada ordenada.

4. Línea de comandos y señales

```
proxy -c <config.toml> [-t] [-w N]
  -c  fichero de configuración TOML (obligatorio)
  -t  solo valida la configuración y sale
  -w  fuerza el número de workers
```

Valida siempre antes de desplegar:

```
$ ./target/release/proxy -t -c demo/proxy.toml
configuración OK: 3 frontends, 8 backends, 2 workers
```

El proxy es un único proceso con un hilo **master** y N hilos **worker** (`proxy-w0` , `proxy-w1` ..., visibles con `top -H`). Las señales se envían al proceso y las atiende el master:

Señal	Efecto
<code>SIGHUP</code>	Recarga la configuración (también se recarga sola al guardar el fichero)
<code>SIGTERM</code> , <code>SIGINT</code>	Parada ordenada: deja de aceptar conexiones y espera hasta 10 s a las peticiones en curso

```
kill -HUP $(pgrep -f 'proxy -c')
```

Si un worker termina por un error interno (un `panic`), solo cae ese hilo: el master lo relanza automáticamente y el resto sigue atendiendo tráfico. Queda registrado en el log (`worker N terminó por un pánico`) y en `worker_restarts` de las estadísticas.

5. Referencia de configuración

La configuración es un fichero **TOML**. Las rutas relativas (certificados, log) se resuelven respecto al propio fichero. Una clave desconocida o mal escrita es un error: el proxy no arranca, o en una recarga la descarta y mantiene la configuración anterior.

5.1 [global]

Clave	Tipo	Por defecto	Descripción
workers	"auto" o 1-128	"auto"	Hilos worker; "auto" = nº de CPUs. Cambiarlo requiere reiniciar
stats_socket	ruta	/tmp/proxy.sock	Socket UNIX de estadísticas (máx. 107 caracteres)
log_file	ruta	stderr	Fichero de log; "-" = stderr
log_level	texto	"info"	debug · info · warn · error
access_log	booleano	true	Una línea por petición

5.2 [timeouts]

Todos en milisegundos.

Clave	Por defecto	Qué controla	Al vencer
client_header	10000	Tiempo máximo para recibir la cabecera (y el handshake TLS)	408
client_idle	60000	Conexión persistente ociosa y túnel WebSocket sin tráfico	cierre
upstream_connect	2000	Conexión a un servidor backend	otro servidor, o 504
upstream_read	30000	Tiempo sin recibir datos del backend	504
upstream_idle	30000	Conexión reutilizable ociosa hacia el backend	cierre

5.3 [[frontend]] y rutas

Cada `[[frontend]]` es un puerto de entrada. Sus rutas se declaran justo debajo con `[[frontend.route]]`.

Clave	Descripción
name	Nombre único (aparece en los logs)
listen	"ip:puerto", "[::]:puerto" o "*:puerto"
tls	true para HTTPS (requiere certs)
certs	Lista de { sni, cert, key } (ver capítulo 7)
default_cert	Índice del certificado para clientes sin SNI o con SNI desconocido (0 por defecto)
route.host	"dominio", "*.dominio" o "default"
route.backend	Nombre de un [[backend]]

```
[[frontend]]
name      = "web"
listen    = "0.0.0.0:8080"

[[frontend.route]]
host      = "api.test"
backend   = "api"

[[frontend.route]]
host      = "default"
backend   = "web"
```

5.4 [[backend]] y health

Clave	Por defecto	Descripción
name	—	Nombre único, referenciado desde las rutas
strategy	round_robin	round_robin · weighted · least_conn · least_load
max_idle	64	Conexiones persistentes ociosas que se guardan por servidor
servers	—	Lista de { addr = "host:puerto", weight = 1 } (1-64 servidores; peso 1-1000)
health.type	sin sección: none	tcp · http · none
health.path	/health	Ruta consultada con type = "http" (espera 2xx)
health.interval	2000	ms entre sondas
health.timeout	500	ms máximos por sonda
health.rise / fall	2 / 3	Éxitos para volver a "up" / fallos para pasar a "down"

```
[[backend]]
name      = "api"
strategy  = "least_load"
servers   = [ { addr = "10.0.0.11:9000" }, { addr = "10.0.0.12:9000" } ]

[[backend.health]]
type      = "http"
path      = "/health"
```

6. Enrutamiento por dominio

Para cada petición se toma el dominio de la cabecera `Host` (sin puerto, sin distinguir mayúsculas) y se busca en las rutas del frontend por el que entró, en este orden:

1. **Dominio exacto:** `api.test` .
2. **Comodín:** `*.example.com` cubre `a.example.com` y `a.b.example.com` , pero no `example.com` . Si varios encajan, gana el más específico (`*.eu.example.com` antes que `*.example.com`).
3. **default**, si existe.
4. Si nada encaja: **404**.

Host pedido	Rutas: <code>api.test</code> · <code>*.example.com</code> · <code>default</code>
<code>api.test</code> , <code>API.TEST:8080</code>	<code>api.test</code>
<code>www.example.com</code>	<code>*.example.com</code>
<code>example.com</code> , <code>otro.test</code>	<code>default</code> (sin default: 404)

Cada petición de una conexión persistente se enruta por separado. El backend recibe además `X-Forwarded-For` , `X-Forwarded-Proto` (`http` / `https`) y `X-Real-IP` con la IP del cliente.

7. HTTPS y certificados

Un frontend con `tls = true` termina HTTPS (TLS 1.2 y 1.3); hacia los backends el tráfico va en HTTP normal. Puedes poner varios certificados en el mismo puerto: se elige según el nombre (SNI) que envía el cliente.

```
[[frontend]]
name = "web-tls"
listen = "0.0.0.0:443"
tls = true
certs = [
  { sni = "api.test", cert = "certs/api.crt", key = "certs/api.key" },
  { sni = "*.example.com", cert = "certs/wild.crt", key = "certs/wild.key" },
]
default_cert = 0
```

- `cert` : certificado en PEM; puede incluir la cadena intermedia a continuación.
- `key` : clave privada en PEM (PKCS#8, PKCS#1 o SEC1), sin contraseña. Se comprueba que corresponde al certificado.
- En `sni` , el comodín cubre una sola etiqueta (`*.example.com` no vale para `a.b.example.com`), igual que en los certificados reales.

Si el cliente pide por SNI un nombre y luego envía un `Host` distinto que su certificado no cubre, recibe **421 Misdirected Request**.

Crear un certificado de prueba

```
openssl req -x509 -newkey rsa:2048 -nodes -days 365 -sha256 \
  -keyout certs/api.key -out certs/api.crt \
  -subj "/CN=api.test" -addext "subjectAltName=DNS:api.test"
```

Comprobar qué certificado sirve el proxy:

```
openssl s_client -connect 127.0.0.1:443 -servername api.test </dev/null 2>/dev/null \
| openssl x509 -noout -subject
```

8. Balanceo de carga

Estrategia	Cómo reparte	Úsala cuando...
round_robin	Por turnos	Los servidores son iguales y las peticiones parecidas
weighted	Proporcional a weight , sin ráfagas	Hay máquinas de distinta capacidad
least_conn	Al que tiene menos peticiones en curso	Algunas peticiones son mucho más lentas que otras
least_load	Al que informa de menos carga	Los servidores pueden medir su propia carga

Cómo informa un servidor de su carga

Con least_load , cada servidor añade a sus respuestas la cabecera X-Backend-Load con un número de 0.0 (libre) a 1.0 (saturado). El proxy la lee, la suaviza para evitar saltos bruscos y la elimina antes de responder al cliente.

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Backend-Load: 0.37
```

Si un servidor deja de enviarla durante más de 5 s, su carga se considera desconocida y se reparte por conexiones activas.

El reparto es por worker: con 8 workers, cada uno aplica la estrategia a sus propias peticiones, y el efecto global es el mismo.

9. Health checks

- **Pasivos** (siempre activos): si conectar a un servidor falla `fall` veces seguidas, deja de recibir tráfico. Sin sondas configuradas, se vuelve a probar pasado `interval`.
- **Activos** (sección `[backend.health]`): el proxy consulta cada servidor cada `interval` ms, por TCP o con `GET path` esperando 2xx. Tras `fall` fallos pasa a "down"; tras `rise` éxitos vuelve a "up".

Cuando un servidor falla al conectar, la petición se reintenta en otro sin que el cliente lo note. Si no queda ninguno sano, el cliente recibe **503**. Los cambios de estado quedan en el log:

```
[warn] [w0] backend 'api' servidor 127.0.0.1:9002 -> DOWN (sonda http)
[warn] [w0] backend 'api' servidor 127.0.0.1:9002 -> UP (sonda http)
```

10. Recarga en caliente

Para cambiar la configuración, **edita y guarda el fichero**: el proxy lo detecta en menos de un segundo (funciona igual con editores que guardan por renombrado, como vim, o con `sed -i`). También puedes forzarla con `kill -HUP`.

- La nueva configuración se valida entera antes de aplicarse. Si tiene un error, **se descarta, se mantiene la actual** y el motivo queda en el log.
- Las peticiones en curso terminan con la configuración anterior; las nuevas usan la nueva. No se corta ninguna conexión.
- Se pueden añadir o quitar frontends (puertos), rutas, backends, servidores y certificados.
- Cambiar `workers` o `stats_socket` requiere reiniciar.

```
[info] [master] configuración recargada (cambio en fichero)
[error] [master] recarga rechazada, se mantiene la configuración actual:
          TOML inválido: TOML parse error at line 136, column 6 ...
```

11. Supervisión: estadísticas y logs

11.1 Estadísticas

```
socat - UNIX-CONNECT:/tmp/proxy.sock | jq .      # o: nc -U /tmp/proxy.sock
```

```
{
  "uptime_s": 1.6, "workers": 2, "workers_alive": 2, "worker_restarts": 0,
  "reloads_ok": 0, "reloads_failed": 0,
  "connections_accepted": 2, "connections_active": 0, "tls_handshakes": 0,
  "requests_total": 10,
  "responses": { "1xx": 0, "2xx": 9, "3xx": 0, "4xx": 1, "5xx": 0 },
  "upstream": { "connects": 3, "reuses": 6, "retries": 0 },
  "buffers_in_use": 0, "log_dropped": 0,
  "backends": [
    { "backend": "api", "server": "127.0.0.1:19001", "state": "up", "up_workers": 2,
      "active": 0, "idle": 1, "load": 0.020, "requests": 3, "failures": 0 }
  ]
}
```

Campo	Significado
<code>upstream.reuses / connects</code>	Peticiones servidas por conexiones reutilizadas / conexiones nuevas a backends. Un ratio alto es lo esperado
<code>upstream.retries</code>	Reintentos por conexiones persistentes que el backend había cerrado
<code>backends[].state</code>	<code>up</code> , <code>down</code> o <code>degraded</code> (los workers no coinciden)
<code>backends[].load</code>	Carga media informada (<code>null</code> si no se conoce)
<code>log_dropped</code>	Mensajes de log descartados porque el disco no daba abasto

11.2 Logs

Formato: fecha [nivel] [hilo] mensaje. El hilo es master o wN (los mensajes de las sondas de salud de un worker llevan también su wN). Cada petición genera una línea de acceso:

2026-09-26T23:12:47.184Z [access] [w0] 127.0.0.1 api.test "GET /hola" 200 127.0.0.1:19001 1ms

- ├── 127.0.0.1
└── IP del cliente
- ├── api.test
└── dominio (Host)
- ├── "GET /hola"
└── método y ruta
- ├── 200
└── código HTTP
- ├── 127.0.0.1:19001
└── servidor que respondió
- └── 1ms
└── latencia

Con mucho tráfico, desactiva el access log (`access log = false`) o usa `log level = "warn"`.

12. Resolución de problemas

Síntoma	Causa probable	Qué hacer
No arranca: <code>clave desconocida 'x'</code>	Errata en la configuración	Corrige el nombre; valida con <code>proxy -t -c ...</code>
No arranca: <code>bind ...: Address already in use</code>	Otro programa usa ese puerto	<code>ss -ltnp grep :8080</code>
No arranca: no se puede cargar el certificado	Ruta mal escrita (es relativa al <code>.toml</code>) o fichero ilegible	Revisa rutas y permisos
la clave no corresponde al certificado	Pareja cert/key equivocada	Revisa la pareja cert/key
Log: <code>stats_socket</code> demasiado largo	Ruta del socket de más de 107 caracteres	Usa una ruta corta, p. ej. <code>/tmp/proxy.sock</code>
404	Ninguna ruta para ese dominio	Añade la ruta o una <code>default</code> ; revisa el <code>Host</code> que envía el cliente
421	SNI y Host no coinciden	Usa el mismo nombre en la URL y en <code>Host</code> , o un certificado que cubra ambos
431	Cabeceras de más de 16 KB (cookies enormes)	Reduce las cabeceras del cliente
502	No se pudo conectar a ningún servidor, o respondió mal	Comprueba que los backends escuchan en <code>addr</code> ; mira <code>failures</code> en las estadísticas
503	Todos los servidores del backend están "down"	Revisa las sondas (<code>path</code> , puerto) y el log de cambios de estado
504	El backend tardó más que <code>upstream_read</code>	Sube el timeout o investiga el backend
408	El cliente no terminó de enviar la cabecera a tiempo	Normal con clientes lentos o escáneres; ajusta <code>client_header</code>
Un cambio de config no se aplica	La recarga se rechazó	Busca <code>recarga rechazada</code> en el log; <code>reloads_failed</code> en las estadísticas
<code>worker_restarts</code> > 0	Un worker terminó por un error interno y se relanzó	Busca <code>terminó por un pánico</code> en el log y el mensaje del pánico en la salida de errores (stderr)
No compila: falta <code>cmake</code>	El proveedor TLS <code>aws-lc-rs</code> lo necesita	Instala <code>cmake</code> o compila con <code>--no-default-features -F ring</code>

13. Herramientas de prueba y benchmark

13.1 Servidor de pruebas: test_backend

```
test_backend --port N --name NOMBRE [--latency-ms N] [--load F | --load-auto]
              [--threads N] [--bind IP] [--drop-every N]
```

Ruta	Respuesta
/ (cualquier otra)	hello from NOMBRE path=... más relleno hasta 100 B
/health	200 "ok"
/echo	Devuelve el cuerpo enviado (también chunked)
/headers	Devuelve las cabeceras recibidas; útil para ver X-Forwarded-*
/sleep/MS , /big/N	Responde tras MS milisegundos / cuerpo de N bytes
/chunked , /close , /nocontent	Respuesta chunked / delimitada por cierre / 204
/ws	WebSocket de eco
/stats	JSON con las conexiones y peticiones recibidas

--load 0.3 fija la cabecera X-Backend-Load ; --load-auto la calcula según las conexiones activas.

13.2 Cliente WebSocket: ws_probe

```
ws_probe --host api.test 127.0.0.1 18080 /ws "hola"
ws_probe --tls --ca demo/certs/ca.pem --sni api.test 127.0.0.1 18443 /ws "hola"
```

13.3 Tests automáticos

```
cargo test                # 43 tests unitarios
./tests/integration/run.sh # 103 comprobaciones de extremo a extremo (~20 s)
./tests/integration/run.sh target/release # 99 con el build release
cargo test -- --ignored    # la batería de integración desde cargo
```

El script de integración genera su propia configuración, lanza 12 servidores y el proxy, comprueba todas las funcionalidades y lo limpia todo al terminar. Con el build de depuración comprueba además que un worker que entra en pánico (se provoca con SIGUSR1, solo en depuración) se relanza sin cortar el tráfico. PORT_BASE=N cambia los puertos y KEEP_WORK=1 conserva los ficheros para inspeccionarlos.

13.4 Benchmark

```
./bench/bench_proxy.sh          # 30 s por escenario
DURATION=10s WORKERS=4 ./bench/bench_proxy.sh # variante rápida
```

El script compila en release, reparte CPUs entre wrk, el proxy y los backends, y deja un informe en bench/results/. Con PROXY_BIN y BACKEND_BIN se pueden medir otros binarios, por ejemplo los de la versión en C, para comparar:

```
BACKEND_BIN=../version_pthread_proxy/build-release/tools/test_backend ./bench/bench_proxy.sh
```

14. Rendimiento y comparativas

Resultados de referencia (8 workers, WSL2, Ryzen 7 7735HS de 16 hilos, respuesta de 100 B, 30 s por escenario, 0 errores en todos los casos). La meta del proyecto es 50.000 req/s por HTTPS.

Escenario	C, procesos	C, hilos	Rust, backend C	Rust (esta versión)
HTTPS, 100 conexiones	223.372	227.498	203.985	201.662
HTTPS, 200 conexiones	210.342	215.434	197.585	191.660
HTTPS, 400 conexiones	173.506	173.364	162.959	163.457
HTTP, 200 conexiones	307.471	341.468	365.279	338.261

Medias de 2-3 ejecuciones por versión (la columna "Rust, backend C" es una sola ejecución); el detalle está en la documentación técnica, capítulo 12, y en `bench/results/`.

- **Procesos frente a hilos (C):** mismo rendimiento en HTTPS; en HTTP los hilos dan un 6-13 % más.
- **Rust frente a C:** en HTTP, con el mismo backend, Rust iguala o supera a C. En HTTPS queda un ~10 % por debajo por la librería TLS (rustls frente a OpenSSL), no por el proxy.
- Todas las versiones superan la meta más de 3 veces. Para sacar el máximo en tu máquina: `build release`, `access_log = false`, `log_level = "warn"` y `workers` igual al número de núcleos que dediques al proxy.